LSEG & Google ADK Market Intelligence Demo

This project is a demonstration of a multi-agent system built with the **Google Agent Development Kit (ADK)** seamlessly orchestrating the **LSEG Model Context Protocol (MCP)** server. The resulting **Cross-Asset Market Intelligence & Valuation Agent** showcases how a team of specialized Large Language Models (LLMs)—including a root orchestrator, a Python-enabled graphing agent, and a reporting agent—can collaboratively reason over live institutional financial data, pricing logic, and news sentiment.

🏗️ Architecture

The architecture illustrates a powerful synergy between the Google ADK and LSEG's specialized financial APIs bridged via standard MCP:

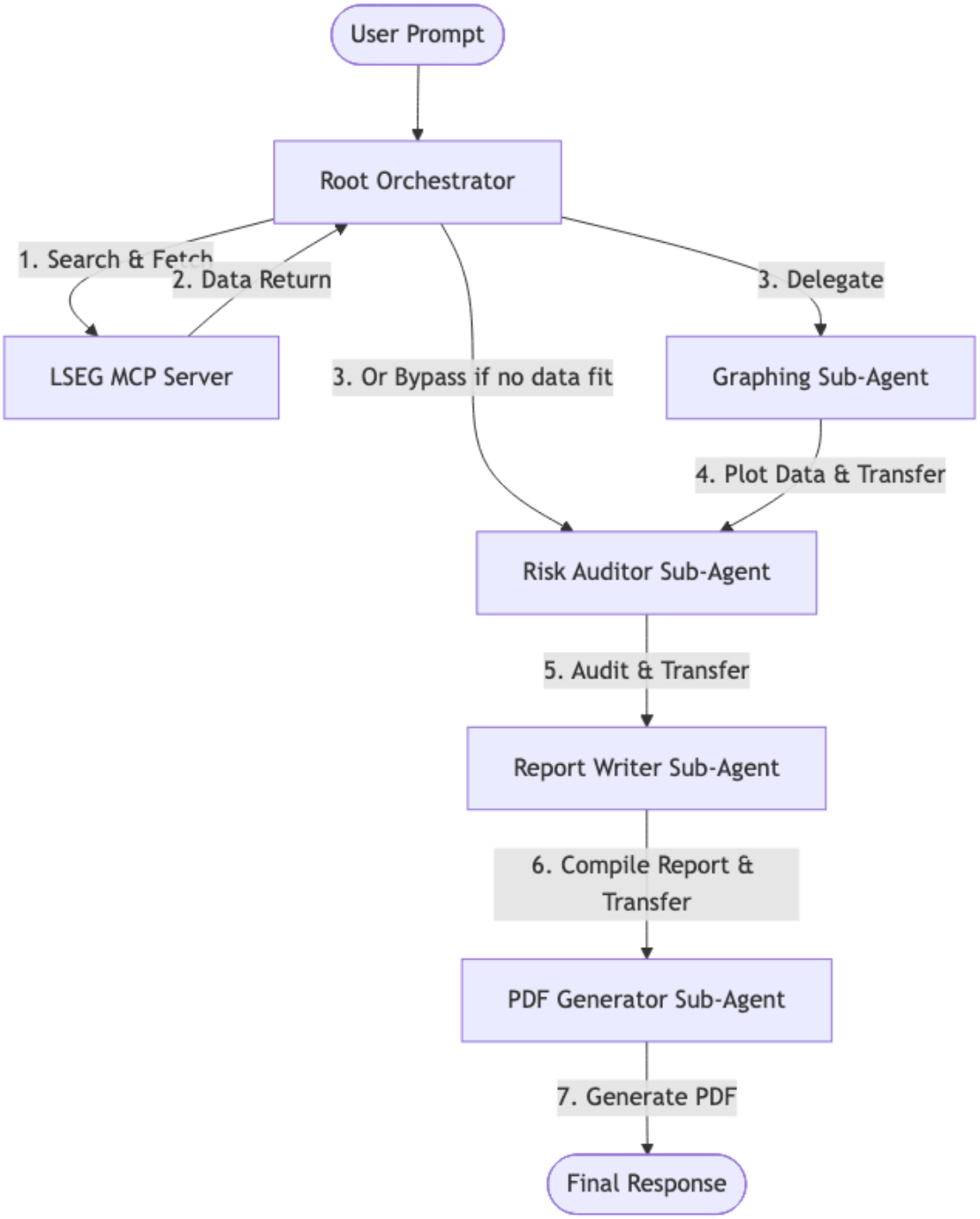
Agent Architecture Diagram

1. **Multi-Agent Orchestration (Google ADK)**: The system is structured using a multi-agent framework:
    - **Root Orchestrator (`lseg_market_agent`)**: Acts as the cognitive orchestration engine. It autonomously queries the LSEG tools, tracks context, and decides which sub-agents to delegate to.
    - **Specialized Skills Agents**:
        - **Equity Research Sub-Agent (`equity_research_agent`)**: Generates comprehensive equity research snapshots combining consensus estimates, fundamentals, price history, and macro data.
        - **Macro Rates Monitor Sub-Agent (`macro_rates_agent`)**: Builds macroeconomic and rates dashboards combining yield curves, inflation breakevens, and swap rates.
    - **Utility Sub-Agents**:
        - **Graphing Sub-Agent (`graphing_agent`)**: Equipped with a Python Code Execution environment (`BuiltInCodeExecutor`) to dynamically generate financial plots, candlestick charts, and visualizations.
        - **Risk Auditor Sub-Agent (`risk_critic_agent`)**: Critiques the gathered financial analysis for over-optimism, evaluates downside risks, and suggests hedging notes.
        - **Report Writer Sub-Agent (`report_agent`)**: Synthesizes all gathered data, risk audits, and visual inferences into a final, comprehensive, and professional Markdown report.
        - **PDF Generator Sub-Agent (`pdf_generator_agent`)**: Compiles the final Markdown report and any visual plots into a professional, downloadable PDF artifact.
2. **MCP HTTP Client Bridge**: Rather than using standard I/O (stdio) proxy executables, this application natively binds to the LSEG HTTP MCP endpoint `https://api.analytics.lseg.com/lfa/mcp` using ADK's `StreamableHTTPConnectionParams` .
3. **LSEG Authentication**: Handled automatically in Python by `mcp_client_bridge.py` , fetching an ephemeral JWT token via OAuth2 client-credentials logic to secure the MCP communication seamlessly.

4. **Tool Discovery & Routing**: ADK maps the structured JSON-schemas from the LSEG MCP discovery phase into the LLM's function-calling toolset allowing the LLM to autonomously retrieve required market data to answer ambiguous questions.

🔄 Multi-Agent Workflow & Interaction

The system takes a pipelined, cascaded approach to answering complex financial analysis queries. The flow of custody ensures that data gathering, visual rendering, compliance auditing, and reporting are executed in a strict, audited sequence:



Key Interaction Mechanisms:

1. **Automated Chain of Custody**: Agents automatically use explicit routing instructions specified in their system prompt to advance the workflow state (e.g., Orchestrator -> Graphing -> Risk Critic -> Report -> PDF). Or directly Orchestrator -> Risk Critic if no visualization is appropriate.
2. **Proactive Visualization**: Even if the user doesn't ask for a graph, the Orchestrator analyzes the quantitative dataset dimensions (e.g., timeseries size or forward estimate spreads) and decides if a graph would make the analysis more impactful, triggers the `graphing_agent` proactively.

3. **Draft Audit compliance**: The final report output does not originate from a single LLM. It involves a descriptive Risk Auditor audit that enforces downside risks or forward over-optimism critiques aren't skipped in the final synthesis conducted by the `report_agent`.

🛠️ LSEG MCP Tools Available

The agent has access to **17 specialized financial tools** served by the LSEG MCP server. It decides autonomously which tools to combine based on the user's prompt:

Pricing & Valuation Tools

- `fx_spot_price`: Calculates FX spot pricing, valuation, and analytics (e.g., EUR/USD).
- `fx_forward_price`: Calculates FX forward pricing and analytics.
- `bond_price`: Prices corporate and sovereign bonds via identifiers (ISIN, CUSIP, RIC).
- `bond_future_price`: Prices bond futures via instrument code.
- `option_value`: Values vanilla, exotic barrier, and binary options with Greeks (Delta, Gamma, Vega).
- `ir_swap`: Prices Interest Rate Swaps based on customizable reference templates.

Market Data Curves & Surfaces

- `interest_rate_curve`: Fetches available IR curves and calculates curve points.
- `credit_curve`: Generates and retrieves corporate and sovereign credit curves.
- `inflation_curve`: Searches and calculates inflation curves by country.
- `fx_forward_curve`: Calculates FX Forward curves.
- `fx_vol_surface`: Generates FX Volatility surfaces.
- `equity_vol_surface`: Generates Equity Volatility surfaces.

Quantitative Analytics (QA) & Fundamentals

- `qa_company_fundamentals`: Retrieves deep historical financials (Net Sales, Gross Income, EPS, DPS, P/E Ratio).
- `qa_ibes_consensus`: Provides forward-looking IBES Analyst Consensus metrics.
- `qa_macroeconomic`: Global economic indicator database (GDP, CPI, Unemployment, Non-Farm Payrolls).

News & Time Series

- `insight_headlines`: Ingests and aggregates the latest news and sentiment for specific companies (RICs) or topics.
- `tscc_interday_summaries`: Consolidates multi-frequency interday pricing summaries (time series data).

🚀 Setup & Installation

Prerequisites:

- Python 3.12+
- LSEG Workspace / Developer Portal API credentials
- Google Cloud CLI (`gcloud`) for Vertex AI access

1. It is recommended to create a virtual environment first:

```
python3.12 -m venv venv
source venv/bin/activate
```

2. Install dependencies:

```
pip install -r requirements.txt
```

3. Clone this repository locally.

Authentication & Configuration

First, authenticate your local environment with Google Cloud so the ADK can access Gemini via Vertex AI:

```
gcloud auth application-default login
```

Set up your `.env` file with your Google Cloud Project ID and LSEG MCP Service Account credentials (generated via the LSEG Workspace / Developer Portal Service Accounts tab):

```
cp .env.example .env
```

Inside `.env`, provide:

```
# Google Cloud Vertex AI
GOOGLE_CLOUD_PROJECT="YOUR_GOOGLE_CLOUD_PROJECT_ID"
GOOGLE_GENAI_USE_VERTEXAI="true"

# LSEG MCP Service Account
LSEG_CLIENT_ID="GE-XX-XXXXXX"
LSEG_CLIENT_SECRET="XXXXX-XXXX-XXXX-XXXXX"
```

Running the Agent

You can interact with the agent natively via the built-in CLI Runner, or spin it up locally in a browser using the `adk web` utility.

Method 1: CLI Runner

Execute the runner directly from standard out using a prompt override:

```
python3.12 run.py --prompt "Compare Apple's latest fundamentals with recent news sentiment."
```

Method 2: ADK Web Interface

Because the application is structured with a root package `lseg_market_agent`, you can spin up the full ADK Gradio UI natively to chat interactively with the agent:

```
adk web .
```

Method 3: Deploy to Vertex AI Agent Engine

You can deploy this agent to Google Cloud's Vertex AI Agent Engine using the standalone ADK CLI. This will package the agent and deploy it to your GCP project as a managed service. *(Ensure you have run `gcloud auth application-default login` and have the Vertex AI API enabled for your project)*:

```
adk deploy agent_engine --project="YOUR_PROJECT_ID" --region="us-central1" --
display_name="LSEG Market Agent" lseg_market_agent
```

Once deployed, you can run and interact with the remote agent directly via Google Cloud or by using the ADK runtime. To run it from the CLI, obtain your Reasoning Engine ID from the deployment output and run:

```
adk run lseg_market_agent --session_service_uri
"agentengine://<YOUR_REASONING_ENGINE_ID>"
```

🧪 Comprehensive Testing Queries

The real power of this integration is the LLM's ability to orchestrate multi-modal tool requests. Paste any of these queries into the CLI or the ADK Web interface to test the agent's capabilities:

Level 1: Focused Data Retrieval

- **Company Fundamentals**: *"What was Microsoft's (MSFT.O) Gross Income and EPS for 2022 and 2023?"*
- **Macroeconomic Data**: *"Show me the latest US GDP and Unemployment figures (search for the Mnemonics if you don't know them)."*
- **News & Sentiment**: *"Get me the latest insight headlines specifically mentioning Tesla (TSLA.O)."*
- **Forward Consensus**: *"Fetch the year-over-year forward analyst earnings expectations for Amazon (RIC=AMZN.O) through 2027."*

Level 2: Cross-Asset Market Intelligence

- *"Analyze Apple's (AAPL.O) recent financial fundamentals, check the latest news sentiment around it, and fetch analyst consensus estimates for the next year to provide a complete investment summary."*
- *"How is the recent Spanish macroeconomic trajectory (CPI and GDP) affecting the construction sector? Get the latest macro stats and cross-index it with news headlines about Spanish Real Estate or Construction."*

Level 3: Pricing & Math Reasoning

- *"Price a vanilla European Call Option for Apple (AAPL.O) expiring on December 31, 2025. Set the strike at $200. What is the Delta and Vega? Note: You may need to guess current standard market parameters."*
- *"I want to execute a €5,000,000 FX Spot trade between EUR and USD. Price the EURUSD cross, then price a 6-month EURUSD FX Forward and compare the implied swap points."*
- *"Can you list the available US Treasury interest rate curves? Pick the most standard one to calculate curve points for."*
- **Credit & Bond Valuation**: *"For a high-leverage company like AT&T (T), inspect its corporate debt rating or credit curves to formulate an overlay regarding credit leverage health and yield spread."*
- **Volatility Surface Assessment**: *"Fetch the equity volatility surface for Microsoft (MSFT.O). Retrieve implied volatility gradients to determine market expectations for short-term downside risks*

(skew profile)."

Level 4: Complex Synthesis & Multi-Agent Delegation

The orchestrator can delegate specialized tasks—such as dynamically generating Python visualizations and formatting Markdown—to its sub-agents to provide a visually compelling, presentation-ready output.

- **Bar Charts & Synthesis**: *"Fetch the historical EPS and Revenue for Amazon over the last 3 years. Draw a grouped bar chart of the EPS vs Revenue data. Then write a final report summarizing the trends."*
- **Trend Lines & Moving Averages**: *"Retrieve Microsoft's interday stock price summary for the last month. Plot the closing prices as a line chart with a 5-day moving average overlaid. Transfer to the report agent for an executive summary."*
- **Candlestick Charts**: *"Get the recent daily price action for Tesla (TSLA.O). Instruct the graphing agent to render this as a candlestick chart showing the open, high, low, and close prices, and format the visual beautifully."*
- **Multi-Metric Scatter Plots**: *"Gather the forward P/E ratios and Dividend Yields for Apple, Microsoft, and Google based on consensus estimates. Create a scatter plot visualizing this relationship with labels for each company, then write a short thesis."*
- **Full Executive Thesis**: *"Act as an institutional portfolio manager. Evaluate Vodafone (VOD.L). Retrieve its historical fundamentals (2020-2023), its forward analyst consensus estimates (2024-2026), its latest news trends, and recent stock price trajectory. Graph the stock trajectory and assemble everything into an executive thesis."*
- **PDF Report Evaluation**: *"Run a complete analysis for Microsoft (MSFT.O) including growth metrics and news sentiment. Explicitly trigger the PDF generator step to produce a downloadable PDF file summarizing everything."*

Level 5: Specialized Skills Workflows

These queries trigger the newly integrated specialized subagents (`equity_research_agent` and `macro_rates_agent`) which follow strict, optimized workflows and output formats (standardized Markdown tables).

- **Equity Research Deep Dive**: *"Perform a deep-dive Equity Research on Tesla (TSLA.O). Use the Equity Research skill workflow: Consensus, Fundamentals, Price, and Macro context."*
- **Macro Rates Monitor**: *"Build a Macroeconomic and Rates Monitor dashboard for the United States. Include GDP/CPI indicators, yield curve snapshot, and real rate decomposition."*
- **Skill Composition (Advanced)**: *"Use the Equity Research skill to analyze Amazon (AMZN.O). Once done, ensure the system generates a professional report and PDF."*

📊 Automated Evaluations (ADK Evals)

The project includes an automated evaluation suite powered by the Google ADK's `AgentEvaluator`. This allows you to verify that the agent is calling the correct tools for specific user prompts without manual interaction.

🔍 What is There?

The `evals/` directory contains JSON-based test cases (e.g., `lseg_market_evals.test.json`, `case_1_msft.test.json`). These files define:

- **`user_content`** : The prompt sent to the agent.

- `intermediate_data.tool_uses` : The expected tool calls (by name) that the agent must make to successfully resolve the prompt.

⚙️ How It Works

The evaluation system uses the `AgentEvaluator.evaluate` method from the Google ADK. It:

1. Spins up the `lseg_market_agent` .
2. Passes the user prompt from the JSON file to the agent.
3. Captures the agent's tool calls and compares them against the expected `tool_uses` defined in the JSON.
4. Reports success (✅ `PASSED`) or failure (❌ `FAILED`) based on whether the expected tools were invoked.

🏃 How to Use It

There are two ways to run the evaluations: using the provided Python script or via the ADK CLI.

Method 1: Using the Python Script

This is the easiest way to run all test cases sequentially:

```
python3.12 run_evals.py
```

Method 2: Using the ADK CLI

If you have the `adk` CLI installed in your environment, you can use the `adk eval` command to run evaluations directly for the `lseg_market_agent` module:

```
adk eval lseg_market_agent evals/case_1_msft.test.json
```

Or run all evaluations by passing multiple files:

```
adk eval lseg_market_agent evals/*.test.json
```

You should see output similar to:

```
Starting evaluations for 'lseg_market_agent'...
Found 3 test files.

--- Running evaluation with dataset: evals/case_1_msft.test.json ---
✅ PASSED evals/case_1_msft.test.json

--- Running evaluation with dataset: evals/case_2_gdp.test.json ---
✅ PASSED evals/case_2_gdp.test.json
...
```